

Assignment 5

2026.2.14

Extension 1: Local sampling

In the standard RRT-Connect algorithm, random samples are drawn uniformly from the entire configuration space. For the PUMA robot with 6 joints, this means sampling across the full range of all joint angles. However, the vast majority of these samples are far from the existing tree and cannot be connected without hitting obstacles. This is especially wasteful in the given problem where the L-shaped end-effector must pass through a narrow hole — the feasible region near the hole is a tiny fraction of the entire configuration space.

Local sampling addresses this by generating samples in the neighborhood of existing tree nodes. Instead of exploring the entire space uniformly, the tree grows by expanding outward from its current frontier. This is analogous to how a plant grows — extending from its existing branches rather than teleporting to random locations.

The implementation consists of two parts: generating a random direction vector in the sampler, and offsetting it from a random tree node in the planner.

Sampler — generateUnit(fac, VOL) in YourSampler.cpp:

```
1 // Generate a random direction vector in configuration space,
2 // normalize it and scale by fac.
3 ::rl::math::Vector sampleq(this->model->getDof());
4 ::rl::math::Vector maximum(this->model->getMaximum());
5 ::rl::math::Vector minimum(this->model->getMinimum());
6
7 for (::std::size_t i = 0; i < this->model->getDof(); ++i)
8 {
9     sampleq(i) = minimum(i) + this->rand()
10        * (maximum(i) - minimum(i));
11 }
12
13 // Compute vector length and normalize
14 ::rl::math::Real len = 0.;
15 for (::std::size_t i = 0; i < this->model->getDof(); ++i)
16 {
17     len += ::std::pow(sampleq(i), 2);
18 }
19 len = ::std::sqrt(len);
20
21 // Scale to sampling radius
22 sampleq = (sampleq / len / (2*M_PI)) * fac;
```

Planner — choose() in YourPlanner.cpp:

```
1 if (ADD)
2 {
3     // Generate a random direction vector
4     chosen = this->sampler->generateUnit(10., ADDVolSampling);
5
6     // Pick a random vertex from the tree
7     u_int64_t len = ::boost::num_vertices(tree);
8     int idx = (int)(len * this->sampler->rand());
9     Vertex vert = ::boost::vertex(idx, tree);
10    rq = *(tree)[vert].q;
11
12    // Offset: direction vector + tree node = local sample
13    chosen += rq;
14    this->model->clip(chosen);
15 }
```

Listing 1: Implementation of Extension 1

The algorithm proceeds in three steps:

Step 1: A random direction vector is generated by sampling a point in joint space, normalizing it, and scaling it by $\text{fac}/(2\pi)$. With $\text{fac} = 10$, this yields a sampling radius of approximately 1.59 radians.

Step 2: A random node is selected from the current tree. The selection is uniform — each node has equal probability of being chosen, regardless of its position or depth in the tree.

Step 3: The direction vector is added to the selected node’s configuration to produce the final sample:

$$q_{\text{sample}} = q_{\text{node}} + \frac{\text{fac}}{2\pi} \cdot \hat{v}_{\text{random}}$$

The result is clipped to joint limits via `model->clip()` to ensure it remains a valid configuration.

Local sampling concentrates the exploration around the existing tree frontier. For the narrow-passage problem (L-shaped end-effector through a small hole), this means the tree incrementally pushes through the hole one small step at a time, rather than hoping a random global sample lands on the other side. The tree grows more densely near obstacles, which increases the chance of finding collision-free paths through tight spaces. Thus, this provides a mitigation against the narrow passage problem.

The trade-off is reduced exploration of distant regions. If the solution requires traversing a large open space, local sampling may be slower than uniform sampling because the tree expands gradually rather than jumping across the space. This is mitigated by combining local sampling with goal-biased sampling (Extension 2).

Extension 2: Goal-biased local sampling

Extension 1 (local sampling) samples around a randomly chosen tree node. While this improves exploration near the tree, it does not provide any directional bias, the tree grows uniformly in all directions from its frontier. In a bidirectional RRT, we know where the other tree is rooted (at the goal or start configuration), so we can direct growth toward the other tree to accelerate the connection.

Goal-biased local sampling addresses this by preferentially sampling around the tree node that is closest to the other tree’s root. This guides the tree’s growth toward the goal region while still maintaining the benefits of local exploration.

The sampling is controlled in `solve()` with a probability parameter `goalbias = 0.2`:

```

1 // In solve(): sampling strategy selection
2 if (this->sampler->rand() > goalbias)
3 {
4     // 80%: local sampling around random tree node
5     this->choose(chosen, *a);
6 }
7 else
8 {
9     // 20%: sample near closest node to other tree's root
10    if (j == 0) target = *this->goal;
11    if (j == 1) target = *this->start;
12    this->chooseN(chosen, *a, target);
13 }

```

The goal-biased sampling function `chooseN()`:

```

1 void YourPlanner::chooseN(::rl::math::Vector& chosen,
2                           Tree& tree,
3                           ::rl::math::Vector& target)
4 {

```

```

5 // Generate a random direction vector (same as Extension 1)
6 chosen = this->sampler->generateUnit(10., ADDVolSampling);
7
8 // Find the node in tree closest to the other tree's root
9 ::rl::math::Vector nq;
10 nq = *tree[this->nearest(tree, target).first].q;
11
12 // Offset the direction vector from that nearest node
13 chosen += nq;
14 this->model->clip(chosen);
15 }

```

Listing 2: Implementation of Extension 2

By directing 20% of samples toward the goal region, the tree grows preferentially in the direction where connection is most likely. This accelerates convergence in problems where the start and goal are far apart in configuration space.

The main trade-off is computational cost: each goal-biased sample requires an additional `nearest()` call to find the closest node to the target, which is a linear scan over all tree vertices. This is why the goal bias probability is kept at 20% rather than a higher value — too frequent `nearest()` calls would become a bottleneck as the tree grows. The 80/20 split balances exploration (Extension 1) with exploitation (Extension 2). Furthermore, it can lead to an end of exploration when the bias is too high because the nearest node could also be within a open convex topology, which would it make impossible to leave it.

Extension 3: Probabilistic early termination

The *Connect* extension already included in the RRTConCon algorithm allows for extending the branch of the tree all the way to the random sample or until a collision occurs. This allows growing the tree quickly and with fewer samples. It also has the benefit of straightening out the branches instead of having each branch oriented in a random direction, allowing for smoother paths.

However, the very explorative nature of this strategy can also be problematic. If a sample is located in the opposite direction of the goal, a lot of collision checks have been wasted on exploring an area which likely will not help in reaching the goal. Even if a sample might be in the correct direction of the goal but far away from it, the tree will grow right past the goal, again wasting collision checks.

This extension intends on improving performance by terminating the extension towards the random sample if said extension does not result in reducing the distance to the goal. However, since some environments require moving away from the goal to find a valid path, this heuristic must not be applied in all cases. In this implementation, termination of an extension away from the goal is done with a probability of 10%.

```

1 ...
2 ::rl::math::Real distanceLastToGoal;
3 ::rl::math::Real distanceNextToGoal;
4
5 if ((&this->tree[0]) == (& tree)){
6 // Connect is in tree a, goal should be b or goal
7 distanceLastToGoal = this->model->transformedDistance(*last, *(this->goal));
8 distanceNextToGoal = this->model->transformedDistance(next, *(this->goal));
9 } else {
10 // Connect is in tree b, goal should be a or start
11 distanceLastToGoal = this->model->transformedDistance(*last, *(this->start));
12 distanceNextToGoal = this->model->transformedDistance(next, *(this->start));
13 }
14
15 // Previous point was closer to goal - stop connect with certain probability

```

```

16 if (distanceLastToGoal < distanceNextToGoal) {
17     if (this->sampler->rand() > 0.90) break;
18 }
19 ...

```

Listing 3: Implementation of Extension 3

The extension is implemented within the extension while-loop of the `connect` function and is very straightforward. After determining which vector is the goal from the view of the current tree, it simply checks whether the next extension would increase the distance to the said goal compared to the current iteration. If so, there is a 10% chance that the while-loop is exited, and thus the extension stopped.

Possible variations of this extension would be to measure the distance to the nearest node of the opposing tree rather than the goal or have the probability of termination determined by how much the distance to the goal would increase. However, both approaches add unnecessary computations and would need manual fine-tuning to work properly.

Extension 4: Intermediate Node Insertion

In the standard RRT-Connect, `connect()` only adds a single vertex at the final position where it stopped. If the connect step traverses a long collision-free path (e.g., 50 delta-steps), no intermediate vertices are recorded — the tree remains sparse along that path. By inserting intermediate vertices during connect, we densify the tree and provide more anchor points for future nearest-neighbor queries.

Inside the connect loop, a counter tracks steps since the last inserted vertex. Every `connectAddIntermediateNode = 10` steps, a new vertex is added:

```

1 int addVertexCounter = 0;
2 while (!reached)
3 {
4     // ... extend step, collision check ...
5
6     // Extension 4: Add intermediate vertex to densify the tree
7     addVertexCounter++;
8     if (connectAddIntermediateNode > 0
9         && addVertexCounter >= connectAddIntermediateNode)
10    {
11        Vertex connected = this->addVertex(tree, last);
12        this->addEdge(nearest.first, connected, tree);
13        addVertexCounter = 0;
14    }
15
16    *last = next;
17 }
18
19 // Add final reached/stopped position as a new vertex
20 Vertex connected = this->addVertex(tree, last);
21 this->addEdge(nearest.first, connected, tree);

```

Listing 4: Implementation of Extension 4

The primary benefit is improved tree coverage. With more vertices distributed along successful connect paths, future samples are more likely to find a nearby tree node, reducing wasted extend/connect attempts. This is especially valuable in combination with local sampling (Extension 1), where the sampling radius is limited, denser trees mean local samples are more likely to fall within reach of an existing node.

The trade-off is increased tree size: more vertices means more time spent in nearest-neighbor search (which is linear in the number of vertices). However, in practice, the improved connectivity typically outweighs the search overhead, since each successful connect reduces the total number of iterations needed to find a solution.

Extension 5: Node exhaustion

At each iteration of the RRTConCon algorithm, an attempt is made to connect the two trees to each other. For that, the nearest node of the opposing tree is determined and then a connection attempt made. However, the nearest node of the opposing tree can be behind an obstacle or even in a dead end, meaning all future connection attempts are also likely to fail.

This extension seeks to alleviate this by tracking how many failed connection attempts occurred with each node. Every time a connection is attempted from a node and a collision occurs, a counter is increased for this node, serving as an indicator how unlikely future connection attempt might be. Later, when the nearest node of the opposing tree is being searched, this counter is also checked. If above a certain threshold, the node is not considered for the selection.

```
1 ...
2 if (this->model->isColliding())
3 {
4     collisionCount[nearest.first]++;
5     recentColl += 1.2; // Increase collision tracking integrator
6     break;
7 }
8 ...
```

Listing 5: Incrementing of counter inside connect function

```
1 for (VertexIteratorPair i = ::boost::vertices(tree); i.first != i.second; ++i.first)
2 {
3     // Skip exhausted nodes that repeatedly collided during connect
4     if (stopUsingNodeAt != 0 && collisionCount[*i.first] >= stopUsingNodeAt) continue;
5 ...
```

Listing 6: Checking if counter reached threshold in nearest function

The implementation is fairly simple, with an unordered map called `collisionCount` being introduced to track the collision count of each node. In the `connect` function, if a collision occurs, said counter is increased for the node, as seen in listing 5. The `nearest` function (listing 6) now includes a simply check whether the threshold, in this case 5, has been reached and glosses over a node if that is the case.

Extension 6: Volume sampling

Extension 1 generates direction vectors for local sampling by normalizing a random vector and scaling it to a fixed length. This means all local samples lie on the surface of a hypersphere around the selected tree node. No samples are generated inside the sphere — the interior is entirely unexplored.

In a 6-dimensional configuration space (6 joints of the PUMA), the volume of a hypersphere grows as r^6 . A thin shell on the surface contains only a tiny fraction of the total volume. By sampling only on the surface, the algorithm misses the vast majority of the neighborhood around each tree node.

Volume sampling addresses this by randomizing the length of the direction vector, so samples are distributed throughout the interior of the sphere rather than only on its surface.

The extension is implemented in `generateUnit()` in `YourSampler.cpp`. After generating and normalizing the direction vector, each component is multiplied by an independent random factor in $[0.1, 1.0]$:

```

1 // Randomize vector length to sample volume instead of sphere surface
2 if (VOL) {
3     for (::std::size_t i = 0; i < this->model->getDof(); ++i)
4     {
5         // Use rand number [0.1, 1] to sample volume instead of circle
6         sampleq(i) = samplevol(i) * sampleq(i);
7     }
8 }

```

Listing 7: Implementation of Extension 6

Where `samplevol(i)` is precomputed as:

```

1 samplevol(i) = this->rand() * 0.9 + 0.1;

```

Without volume sampling (`VOL = false`), all samples lie on the surface at a fixed radius $\frac{\text{fac}}{2\pi}$ from the tree node:

$$q_{\text{sample}} = q_{\text{node}} + \frac{\text{fac}}{2\pi} \cdot \hat{v}$$

With volume sampling (`VOL = true`), each component i is independently scaled:

$$q_{\text{sample}}^{(i)} = q_{\text{node}}^{(i)} + \frac{\text{fac}}{2\pi} \cdot \hat{v}^{(i)} \cdot s_i, \quad s_i \sim \mathcal{U}(0.1, 1.0)$$

The lower bound of 0.1 prevents samples from being generated too close to the tree node itself, where they would provide little benefit (the node is already in the tree).

Note that each component uses an *independent* scaling factor s_i , so the resulting distribution is not a uniform ball but rather a randomly stretched ellipsoid. In practice, this provides sufficient coverage of the node’s neighborhood.

Volume sampling significantly increases the coverage of each node’s neighborhood. In 6D configuration space, the ratio of volume to surface is substantial, by filling in the interior, we provide many more candidate configurations for the planner to connect to.

The computational overhead is negligible: it requires only 6 additional random number generations and 6 multiplications per sample. There is no downside in terms of tree size or search time, the number of samples generated per iteration remains the same, only their spatial distribution changes.

This extension works synergistically with Extension 1 (local sampling) and Extension 4 (intermediate nodes): local sampling defines *where* to sample, volume sampling defines the *spread* of those samples, and intermediate nodes ensure the tree is dense enough to benefit from nearby samples.

1 Performance

As can be seen in table 1, the performance gain with our extensions is immense. The average computation time and the standard deviation of the computation time are reduced by a factor of 8, while the average number of nodes is roughly halved. The average number of queries reduces tenfold, at least for the non-reversed case.

Perhaps the most surprising finding is the increased performance when running the RRTConCon algorithm in reverse. Since the algorithm uses two trees growing from both start and goal positions, it would be expected, that it performs roughly equally in both directions. This is the case for the algorithm with our extensions, although it does perform slightly worse in the reversed direction.

	RRTConCon	RRTConCon (reversed)	with Extensions	with Extensions (reversed)
avgT in s	63.3	35.8	7.3	11.8
stdT in s	64.6	27.2	5.3	8.5
avgNodes	11863	8483	6571	8511
avgQueries	585082	436869	53110	68335

Table 1: Performance of the algorithm without and with the extensions

2 Ablation Study

To determine which extension has the greatest influence on the performance gain, each extension is removed from the algorithm one by one, and another 10 benchmark runs are performed. The resulting decrease in performance of the algorithm as a whole can be used as a metric for the performance of the removed extension.

	avgNodes	avgQueries	avgT in s	stdT in s	Ratio to Baseline (avgT)
Baseline (all Extensions)	6571	53110	7.3	5.3	1
w/o Local Sampling (1)	36620	301255	239.5	169.0	32.7
w/o Prob. Goal Bias (2)	9612	78952	22.0	35.0	3.0
w/o Early Termination (3)	11675	107748	13.0	9.5	1.8
w/o Densification (4)	3766	63795	6.4	5.6	0.9
w/o Exhaustion (5)	4861	41027	3.0	2.1	0.4
w/o Volume Sampling (6)	7381	60860	9.7	9.0	1.3

Table 2: Performance of the algorithm without specific extensions

It is clear from the results shown in table 1, that local sampling has a great influence on the performance of the algorithm. Interestingly, the performance without local sampling worsened way beyond the regular RRTConCon algorithm, being about 4 times slower. This suggests that one or more of the extensions is highly reliant on local sampling to reach acceptable performance, and without it actually decreases the performance greatly. This skews the results slightly, but it is safe to say that local sampling plays a huge role in the performance.

Apart from that, probabilistic goal bias and early termination have the biggest influence, with performance worsening threefold and twofold, respectively, with the extensions absent. This is expected since both of these extensions aim to greatly reduce the area exploration of the configuration space and rather focus on areas between the start and goal configurations.

Perhaps the most important conclusion from the ablation study is that the algorithm actually improves in performance by removing some extensions.

Tree densification reduces the performance slightly, which suggests that possible scenarios which would have benefited most from tree densification are already covered well by other extensions like early termination and local sampling. Although the influence on computing time is quite small, it almost doubles the number of average nodes and should thus be removed in future versions.

The most adverse effect on performance is seen with the node exhaustion, performance improving more than twice without it. The idea was to reduce the number of connection

attempts of nodes which are in dead ends or behind walls and thus unreachable. An oversight in this line of thought is, however, that nodes inside narrow passages also could very quickly be exhausted, while still being crucial for connecting the two trees. Since the particular scenario used here is very much a narrow passage problem, it could be the reason for the performance decrease when using this extension.